

3.1 Cover Page

SMART INDUSTRIAL EQUIPMENT MAINTENANCE & PREDICTIVE MONITORING SYSTEM

Data Structures and Algorithms Project Report

Submitted By:

- Name: Shankar Mupanna
- Roll Number:150096725101
- Program: B.Tech Computer Science (AI & ML)
- Institute: ITM Skills University, Kharghar

Academic Year: 2025-26

3.2 Student Details

Student Name: Shankar Mupanna

Course: B.Tech Computer Science (AI & ML)

University: ITM Skills University

Semester: II

Project Type: Data Structures and Algorithms Mini Project

3.3 Project Title

Smart Industrial Equipment Maintenance & Predictive Monitoring System

3.4 Problem Statement

Industrial facilities contain numerous machines that require regular monitoring and maintenance. Manual tracking of machine health, maintenance requests, equipment connectivity, and technician routing can be inefficient and error-prone.

The objective of this project is to develop a maintenance management system that stores machine records, predicts maintenance urgency, manages service requests, tracks historical modifications, models machine connectivity, and determines optimal technician routes using Data Structures and Algorithms.

3.5 Objectives

1. Store and manage machine records efficiently.
 2. Monitor machine health using fault-based calculations.
 3. Handle maintenance requests using FIFO scheduling.
 4. Maintain modification history with undo functionality.
 5. Prioritize machines requiring urgent maintenance.
 6. Implement searching techniques for quick machine retrieval.
 7. Model factory floor connectivity using graphs.
 8. Perform BFS and DFS graph traversals.
 9. Find shortest technician routes using Dijkstra's Algorithm.
 10. Demonstrate practical applications of major DSA concepts.
-

3.6 Design / Architecture

The project is divided into multiple modules:

Module 1 – Machine Management

- Machine Class
- Binary Search Tree (BST)
- Hash Map

Purpose:

- Store machine information.
- Fast insertion, deletion, and retrieval.

Module 2 – Health Monitoring

Health Formula:

Health Score = $100 - (\text{Fault Count} \times 10)$

Status Categories:

- Healthy (>70)
- Maintenance Soon ($41-70$)
- Critical (≤ 40)

Module 3 – Maintenance Queue

Data Structure:

- Queue (FIFO)

Purpose:

- Process maintenance requests in order of arrival.

Module 4 – Undo History

Data Structure:

- Stack (LIFO)

Purpose:

- Restore previous machine states.

Module 5 – Priority Scheduling

Algorithm:

- Merge Sort

Purpose:

- Rank machines by maintenance urgency.

Module 6 – Searching

Algorithms:

- Linear Search
- Binary Search
- Hash Map Search

Purpose:

- Retrieve machine records efficiently.

Module 7 – Connectivity Graph

Data Structure:

- Adjacency List Graph

Purpose:

- Represent machine network layout.

Module 8 – BFS Traversal

Purpose:

- Analyze outage spread across connected machines.

Module 9 – DFS Traversal

Purpose:

- Inspect dependency chains among machines.

Module 10 – Dijkstra Algorithm

Purpose:

- Find shortest technician route to a target machine.

Architecture Flow:

User



System Coordinator



| BST | Queue | Stack | Graph | Sorting |



Output / Reports / Analysis

3.7 Algorithm Description

Binary Search Tree (BST)

Used for machine storage.

Rules:

- Smaller IDs → Left Subtree
- Larger IDs → Right Subtree

Operations:

- Insert
 - Delete
 - Search
 - Inorder Traversal
-

Queue

Used for maintenance requests.

FIFO Principle:

First Request Entered

↓

First Request Processed

Operations:

- Enqueue
 - Dequeue
 - Front
-

Stack

Used for undo operations.

LIFO Principle:

Last Modification

↓

First Modification Reverted

Operations:

- Push
 - Pop
 - Top
-

Merge Sort

Used to rank machines by priority.

Steps:

1. Divide array into halves.
2. Recursively sort both halves.
3. Merge sorted halves.

Priority Formula:

Priority = (100 - Health Score) + (Fault Count × 5)

Linear Search

Checks every machine sequentially.

Suitable for:

- Name-based searching.
-

Binary Search

Works on sorted data.

Procedure:

1. Find middle element.
 2. Compare with target.
 3. Search left or right half.
 4. Repeat until found.
-

Breadth First Search (BFS)

Uses Queue.

Traversal Pattern:

Level by Level

Purpose:

- Outage spread analysis.

Complexity:

$O(V + E)$

Depth First Search (DFS)

Uses Recursion / Stack.

Traversal Pattern:

Go Deep First, Then Backtrack

Purpose:

- Dependency path inspection.

Complexity:

$O(V + E)$

Dijkstra Algorithm

Used for shortest path calculation.

Steps:

1. Initialize distances.
2. Select nearest unvisited node.
3. Relax neighboring edges.
4. Repeat until destination is reached.

Purpose:

- Technician route planning.
-

3.8 Complexity Analysis

Operation	Data Structure / Algorithm	Time Complexity
Insert Machine	BST	$O(\log N)$ Avg
Search Machine	BST	$O(\log N)$ Avg
Delete Machine	BST	$O(\log N)$ Avg
Hash Map Search	unordered_map	$O(1)$ Avg
Queue Enqueue	Queue	$O(1)$

Queue Dequeue	Queue	$O(1)$
Stack Push	Stack	$O(1)$
Stack Pop	Stack	$O(1)$
Linear Search	Array/Vector	$O(N)$
Binary Search	Sorted Array	$O(\log N)$
Merge Sort	Divide & Conquer	$O(N \log N)$
BFS	Graph	$O(V + E)$
DFS	Graph	$O(V + E)$
Dijkstra	Graph	$O(V^2)$

Where:

N = Number of Machines

V = Number of Vertices

E = Number of Edges

3.9 Screenshots of Execution

Insert the following screenshots:

1. Program Startup Screen

```
shankarmupanna@shankars-MacBook-Air Final_Project % cd "/Users/shankarmupanna/Desktop/c++_codes/DSA_1/Final_Project/" && g++ main.cpp -o main && "/Final_Project/"main
=====
                WELCOME TO SMART INDUSTRIAL EQUIPMENT MAINTENANCE & PREDICTIVE MONITORING
=====
Sample data loaded successfully. Ready for DSA Viva demonstration.

=====
                MAIN MENU
=====
1. Add Machine
2. Search Machine
3. Update Machine
4. Delete Machine
5. Display Machines
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14):
```

2. Add Machine Operation

```
shankarmupanna@Shankars-MacBook-Air Final_Project % cd "/Users/shankarmupanna/Desktop/c++_codes/DSA_1/Final_Project/" && g++ main.cpp -o main && "/Final_Project/"main
14. Exit
=====
Select Option (1-14): 1

--- Add New Machine ---
Enter Machine ID: 233
Enter Machine Name: rskfduyg
Enter Location: rkgf
Enter Fault Count: 23
Machine successfully added.
ID: 233 | Name: rskfduyg | Location: rkgf | Health: 0% [Critical] | Faults: 23

=====
                MAIN MENU
=====
1. Add Machine
2. Search Machine
3. Update Machine
4. Delete Machine
5. Display Machines
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14):
```

3. Search Machine Operation

```
=====
MAIN MENU
=====
1. Add Machine
2. Search Machine
3. Update Machine
4. Delete Machine
5. Display Machines
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14): 2

--- Search Machine ---
1. O(1) Hash Map Search (Standard)
2. Linear Search by Name (O(N))
3. Binary Search by ID (O(log N))
Enter Choice: 2
Enter Machine Name: rskfdyug

[O(N) Linear Search Result] Found:
ID: 233 | Name: rskfdyug | Location: rkgf | Health: 0% [Critical] | Faults: 23
⊗ 0 ▲ 0
```

4. Update Machine Operation

```
14. Exit
=====
Select Option (1-14): 3

--- Update Machine Details ---
Enter Machine ID to Update: 101
Enter New Name: jdrsifgc
Enter New Location: arskfydg
Enter New Fault Count: 90
Machine details updated successfully.
ID: 101 | Name: jdrsifgc | Location: arskfydg | Health: 0% [Critical] | Faults: 90
=====
MAIN MENU
⊗ 0 ▲ 0
```

5. Delete Machine Operation

```
10: 101 | Name: jdfsyge | Location: dskfyg | Health: 60
=====
MAIN MENU
=====
1. Add Machine
2. Search Machine
3. Update Machine
4. Delete Machine
5. Display Machines
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14): 4

--- Delete Machine ---
Enter Machine ID to Delete: 102
Machine ID 102 deleted from the system.
```

6. Maintenance Queue Display

```
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14): 6

--- Add Maintenance Request ---
Enter Machine ID: 103
Enter Issue Description: rkfdvugh
Maintenance request successfully registered (FIFO Queue).
Req ID: 503 | Machine ID: 103 | Priority Score: 90 | Issue: rkfdvugh

=====
MAIN MENU
=====
```

7. Undo Operation

```
=====
MAIN MENU
=====
1. Add Machine
2. Search Machine
3. Update Machine
4. Delete Machine
5. Display Machines
6. Add Maintenance Request
7. Process Maintenance Request
8. Undo Last Update
9. Sort Machines By Priority
10. Connect Machines
11. Run BFS
12. Run DFS
13. Run Dijkstra
14. Exit
=====
Select Option (1-14): 8

[UNDO] Rolling back last operation (DELETE)...
Undo successful. Restored Machine State:
ID: 102 | Name: Robotic Welding Arm | Location: Assembly Line 1 | Health: 100% [Healthy] | Faults: 0
=====
```

8. Priority Scheduling Output

```
14. EXIT
=====
Select Option (1-14): 9

=====
MAINTENANCE SCHEDULE (MERGE SORT PRIORITY)
=====
Rank    Priority  ID    Machine Name          Status    Health
-----
1        550       101   jdrsylgc               Critical  0%
2        215       233   rskfdyug               Critical  0%
3        90        103   Hydraulic Press        Critical  40%
4        60        104   Injection Molding MachineMaintenance Soon60%
5        15        105   Industrial Air CompressorHealthy    90%
6        0         102   Robotic Welding Arm    Healthy    100%
=====
```

9. BFS Traversal Output

```
ShankarMupanna@Shankar's-MacBook-Air:Final_Project % cd /Users/ShankarMupanna/
Final_Project/"main

--- Run BFS Outage Spread propagation ---
Enter Start Node ID (0 for Tech Room): 101

BFS Outage Spread propagation (Starting at ID 101):
Step 1: Node ID 101 (jdrsylgc)
Step 2: Node ID 0 (Technician Room)
Step 3: Node ID 103 (Hydraulic Press)
Step 4: Node ID 105 (Industrial Air Compressor)
Step 5: Node ID 104 (Injection Molding Machine)

=====
MAIN MENU
=====
```

10. DFS Traversal Output

11. Dijkstra Shortest Path Output

```
shankarilapanna@shankar's MacBook Air: Final_Project % cd /Users/shankarilapanna/Desktop/C++_codes/DSA_1/Final_Project/ && g++ ma
/Final_Project/"main
14. Exit
=====
Select Option (1-14): 13

--- Run Dijkstra Route Path Planner ---
Enter Target Machine ID to route: 103

=====
DIJKSTRA ROUTE PATH PLANNER
=====
Shortest Path Distance: 45 meters

Step-by-Step Path:
  Technician Room (ID: 0) -> jdrsyfgc (ID: 101) -> Hydraulic Press (ID: 103)
=====

=====
MAIN MENU
=====
0 ^ 0
```

3.10 Results and Findings

The system successfully demonstrated the practical implementation of multiple Data Structures and Algorithms in a single industrial application.

Key Findings:

- BST provided efficient machine record management.
- Queue ensured fair maintenance scheduling.
- Stack enabled reliable undo functionality.
- Merge Sort efficiently prioritized maintenance tasks.
- Hash Map provided near constant-time lookup.
- BFS effectively traced outage propagation.
- DFS successfully analyzed dependency paths.
- Dijkstra generated shortest technician routes.

The project successfully integrates DSA concepts with a real-world industrial maintenance scenario.

3.11 Conclusion

The Smart Industrial Equipment Maintenance & Predictive Monitoring System successfully demonstrates the application of core Data Structures and Algorithms in solving real industrial problems.

The project integrates:

- Binary Search Tree
- Queue
- Stack
- Merge Sort
- Linear Search
- Binary Search
- Graph
- BFS
- DFS
- Dijkstra Algorithm

The system improves machine monitoring, maintenance scheduling, route planning, and operational efficiency while providing a strong practical understanding of DSA concepts.

3.12 GitHub Repository Link

GitHub Repository:

<https://github.com/shankara11y/DSA-CPP-Final-Project>